

ASSESSMENT TOOL-CO3-AT1

EXPERIMENT

Name: S. Bhuvanesh

Reg No: 192521004

Question 5: Analyze the trade-offs between Merge Sort and Quick Sort by comparing their execution time, space complexity, and stability through experimental implementation. Interpret the results and justify which algorithm is more suitable for large-scale applications under different data conditions.

1. Algorithm

Merge Sort

1. Divide the array into two halves.
2. Sort both halves recursively.
3. Merge the sorted halves.
4. Display execution time.

Quick Sort

1. Select a pivot.
2. Partition the array around the pivot.
3. Sort both parts recursively.
4. Display execution time.

2. Pseudocode

Merge Sort

MergeSort(A, low, high)

if low < high

 mid = (low + high)/2

 MergeSort(A, low, mid)

 MergeSort(A, mid+1, high)

 Merge(A, low, mid, high)

Quick Sort

QuickSort(A, low, high)

if low < high

 p = Partition(A, low, high)

 QuickSort(A, low, p-1)

 QuickSort(A, p+1, high)

3. C Program

```
#include <stdio.h>
```

```
#include <time.h>
```

```
void merge(int a[],int l,int m,int r){
```

```
    int i=l,j=m+1,k=0,t[50];
```

```
    while(i<=m&& j<=r)t[k++]=(a[i]<a[j])?a[i++]:a[j++];
```

```
    while(i<=m)t[k++]=a[i++];
```

```

    while(j<=r)t[k++]=a[j++];

    for(i=l,k=0;i<=r;i++)a[i]=t[k++];
}

void ms(int a[],int l,int r){
    if(l<r){
        int m=(l+r)/2;

        ms(a,l,m); ms(a,m+1,r); merge(a,l,m,r);
    }
}

int part(int a[],int l,int r){
    int p=a[r],i=l-1,t;

    for(int j=l;j<r;j++)
        if(a[j]<=p){t=a[++i];a[i]=a[j];a[j]=t;}

    t=a[i+1];a[i+1]=a[r];a[r]=t;

    return i+1;
}

void qs(int a[],int l,int r){
    if(l<r){
        int p=part(a,l,r);

        qs(a,l,p-1); qs(a,p+1,r);
    }
}

int main(){
    int n,a[50],b[50];

```

```

clock_t s,e;

printf("Enter n: ");

scanf("%d",&n);

printf("Enter elements: ");

for(int i=0;i<n;i++) scanf("%d",&a[i]),b[i]=a[i];

s=clock(); ms(a,0,n-1); e=clock();

printf("Merge Sort Time = %f\n",(double)(e-s)/CLOCKS_PER_SEC);

s=clock(); qs(b,0,n-1); e=clock();

printf("Quick Sort Time = %f\n",(double)(e-s)/CLOCKS_PER_SEC);

printf("Sorted: ");

for(int i=0;i<n;i++) printf("%d ",a[i]);

}

```

4. Sample Input

Enter n: 8

Enter elements: 38 27 43 3 9 82 10 5

5. Manual Process

Merge Sort:

38 27 43 3 9 82 10 5

→ 27 38 3 43 9 82 5 10

→ 3 27 38 43 5 9 10 82

→ 3 5 9 10 27 38 43 82

Quick Sort:

Pivot = 5

→ 3 5 43 38 9 82 10 27

→ 3 5 9 10 27 38 43 82

program execution:

```
main.py
1 import time
2 import random
3
4 def merge_sort(arr):
5     if len(arr) <= 1:
6         return arr
7     mid = len(arr) // 2
8     left = merge_sort(arr[:mid])
9     right = merge_sort(arr[mid:])
10    return merge(left, right)
11
12 def merge(left, right):
13     result = []
14     i = j = 0
15     while i < len(left) and j < len(right):
16         if left[i] <= right[j]: # <= ensures stability
17             result.append(left[i])
18             i += 1
19         else:
20             result.append(right[j])
21             j += 1
22     result.extend(left[i:])
23     result.extend(right[j:])
24     return result
25
26 def quick_sort(arr, low, high):
27     if low < high:
```

```

28         pi = partition(arr, low, high)
29         quick_sort(arr, low, pi - 1)
30         quick_sort(arr, pi + 1, high)
31
32     def partition(arr, low, high):
33         pivot = arr[high]
34         i = low - 1
35         for j in range(low, high):
36             if arr[j] < pivot:
37                 i += 1
38                 arr[i], arr[j] = arr[j], arr[i]
39         arr[i + 1], arr[high] = arr[high], arr[i + 1]
40         return i + 1
41
42     if __name__ == "__main__":
43         sample_input = [38, 27, 43, 3, 9, 82, 10]
44         print(f"Original Array: {sample_input}\n")
45
46         # Merge Sort Run
47         start = time.perf_counter()
48         m_res = merge_sort(sample_input.copy())
49         m_time = time.perf_counter() - start
50         print(f"Merge Sort Output: {m_res} (Time: {m_time:.6f}s)")
51
52         # Quick Sort Run
53         q_res = sample_input.copy()
54         start = time.perf_counter()
55         quick_sort(q_res, 0, len(q_res) - 1)
56         q_time = time.perf_counter() - start
57         print(f"Quick Sort Output: {q_res} (Time: {q_time:.6f}s)")

```

Output:

```

Original Array: [38, 27, 43, 3, 9, 82, 10]

Merge Sort Output: [3, 9, 10, 27, 38, 43, 82] (Time: 0.000016s)
Quick Sort Output: [3, 9, 10, 27, 38, 43, 82] (Time: 0.000008s)

```